COMPONENT BASED SOFTWARE ENGINEERING

Component Selection, Qualification and Adaptation

Component repositories

- A component repository is a **centralized, organized store of reusable components and their related information**, such as specifications, documentation, test results, and usage history. It supports the **identification (search) and retrieval** of components, making reuse practical instead of ad hoc.
- Without a repository, developers often don’t know what already exists, so they rebuild components that could have been reused.
- **Why repositories are needed**
  - Make reusable components **discoverable**
  - Reduce duplicated development effort
  - Store **metadata** needed for qualification and selection
  - Manage **versions** and dependencies
  - Improve quality by sharing tested, certified components
  - Support governance, licensing, and security control

component selection criteria

- Selection is the process of identifying and choosing the best candidate component from available internal or external sources.
- It is the activity of identifying, evaluating, and choosing the reusable components (COTS, open source, or in-house) that best satisfy a system’s requirements and architecture.
- It is one of the core steps in component-based development, because a poor choice affects integration effort, quality, cost, and long-term maintenance.
- The goal is to find a component that matches the system's requirements while minimizing cost and development time.

Cont.:

- **Requirements Matching:** Evaluating functional requirements (what the component does) and non-functional requirements (performance, scalability, security, and license constraints).
- **Market Analysis:** Identifying available open-source, commercial, or legacy in-house alternatives.
- **Viability Assessment:** Checking the vendor's reputation, community support (for open source), documentation quality, and release history to avoid abandoned software.
- **Trade-Off Analysis:** Balancing the cost of buying/adopting vs. building from scratch, alongside long-term maintenance implications.

The Selection Process

- Component selection is intertwined with architectural design. The choice of architecture affects which components can be used, and the availability of components can influence architectural decisions .

A structured approach typically follows these stages:

- **Requirement Analysis**: Capture system requirements and define the component requirements needed to meet them .
- **Make-or-Buy Decision**: Determine whether to develop a custom component or reuse an existing one. This strategic decision is typically made early to minimize rework .
- **Candidate Identification**: Search component libraries or marketplaces using functional attributes (what the component does) .
- **Filtering & Evaluation**: Reduce the candidate set using condition attributes (implementation specifics) and then evaluate the survivors against a detailed set of criteria .
- **Integration & Verification**: Integrate selected components and verify that the composed system behaves as intended

Common challenges

- Incomplete or inaccurate vendor documentation
- Mismatch between component interfaces and system needs (architectural mismatch)
- No access to source code, which makes testing and debugging hard
- Vendor lock-in and unexpected end-of-life
- Conflicting criteria, such as the cheapest option being the least reliable

Evaluation Criteria

- Evaluation criteria are the measurable standards used to judge whether a candidate component is suitable. They are usually grouped into categories, and each is given a weight based on project priorities.
- **1. Functional criteria**
- **Functional completeness:** Does it cover all required features?
- **Functional correctness:** Does it produce accurate results under the stated rules?
- **Appropriateness:** Does it avoid unneeded features that add complexity?

2. Quality criteria (based on ISO/IEC 25010)

ATTRIBUTE	WHAT TO EVALUATE
Reliability	Maturity, failure rate, fault tolerance, recoverability
Performance efficiency	Response time, throughput, memory and CPU usage
Security	Authentication, encryption, vulnerability history, patch frequency
Maintainability	Modularity, code quality, ease of change and testing
Portability	Runs across required platforms, OS, and environments
Usability	Ease of learning, API clarity, configuration effort
Compatibility	Interoperability and co-existence with other components

3. Architectural and integration criteria

- Quality and clarity of **provided and required interfaces**
- Support for standards and protocols (REST, SOAP, JDBC, etc.)
- **Coupling and cohesion** of the component
- Degree of **architectural mismatch** with the system
- Effort required for wrappers, adapters, or glue code
- Customizability and extensibility (configuration, plug-ins)

4. Non-technical (business) criteria

Criterion	Questions to ask
Cost	License fees, training, integration, maintenance (total cost of ownership)
Licensing	Proprietary vs. open source, legal restrictions, redistribution rights
Vendor viability	Reputation, financial stability, longevity
Support	SLAs, response times, community activity, update roadmap
Lock-in risk	How hard is it to replace later?

5. Documentation and testability criteria

- Completeness and accuracy of documentation
- Availability of examples, tutorials, and test suites
- Certification or third-party evaluation results
- Ability to test the component in isolation

6. Risk criteria

- Black-box behavior with no source access
- Dependence on third-party updates or hidden dependencies
- Security exposure and compliance issues (e.g., GDPR)
- Risk of end-of-life or abandonment

Component qualification

- Component qualification is the process of determining whether a pre-existing component (often Commercial Off-The-Shelf, or COTS) is "fit for use" in a specific system context .
- It goes beyond simple selection: selection identifies candidates, while qualification proves a chosen candidate can be safely and reliably integrated into your architecture.
- The fundamental goal is to ensure the component satisfies system requirements with high quality and reliability .
- This is particularly critical in safety-critical domains like aerospace, automotive, and nuclear power, where an unqualified component could lead to catastrophic failure .
- Standards like DO-178C (avionics) and ISO 26262 (automotive) mandate rigorous qualification for software libraries and components used in critical paths

Qualification activities

1. **Documentation review:** Check specifications, interface descriptions, and known limitations.
2. **Vendor and source assessment:** Reputation, support, update history, and license terms.
3. **Interface compatibility analysis:** Compare the component's interfaces with the architecture's needs.
4. **Prototyping and trial integration:** Build a small test integration to expose mismatches early.
5. **Testing and benchmarking:** Functional tests, performance tests, and security checks, often in a sandbox.
6. **Risk assessment:** Identify black-box risks, hidden dependencies, and lock-in.
7. **Decision:** Mark the component as qualified, qualified with adaptation, or rejected.

Aspects examined during qualification

Aspect	What is checked
Application programming interface (API)	Are the provided and required interfaces complete, well-defined, and compatible?
Development and integration tools	What tools, languages, and platforms are needed to use it?
Run-time requirements	Resource usage, memory, performance, and response time
Services required	Dependence on OS services, middleware, databases, network, or other components
Security features	Access control, authentication, encryption, vulnerabilities
Embedded design assumptions	Hidden assumptions about data formats, error handling, concurrency, or environment
Exception handling	How errors and failures are reported and recovered from

Possible outcomes

- **Qualified:** Meets requirements as is.
  - **Conditionally qualified:** Usable after adaptation (wrappers, glue code, configuration).
  - **Not qualified:** Fails mandatory criteria and is rejected.

Challenges

- Incomplete or inaccurate documentation
- No source code access for inspection
- Architectural mismatch discovered late
- Limited vendor information or test evidence
- Time and cost of thorough testing

Component Adaptation

- Component adaptation is the process of modifying or adjusting a software component so that it can work correctly with other components or meet the requirements of a particular software system.
- It is necessary when a reusable component does not fully match the system's required interfaces, data formats, behavior, or operating environment.
- Example: A mobile application uses a payment component that accepts amounts in US dollars, but the application operates in Kenyan shillings. The component may need to be adapted to handle the required currency format.

Why it is needed

- **Interface mismatch:** different method names, parameters, or data formats
- **Functional gaps:** missing features or extra unwanted behavior
- **Architectural mismatch:** conflicting assumptions about control flow, data sharing, or concurrency
- **Platform or protocol differences:** e.g., SOAP vs. REST, different data types
- **Non-functional gaps:** performance, security, or logging requirements not met

Component Mismatches

A mismatch is a **conflict between what a component offers or assumes and what the system or other components expect**. Mismatches are the main reason adaptation is needed, and they are the main cause of integration cost and failure in CBA In a perfect world, software components would plug into each other like Lego bricks. In reality, components often suffer from **mismatches** because they were built by different vendors, at different times, or for different systems. These mismatches generally fall into three categories:…….

Cont.: Component Mismatches

These mismatches generally fall into three categories:

- **Signature Mismatches:** Differences in method names, data types, or the number of parameters.
- **Behavioral Mismatches:** Differences in the expected order of operations or execution logic.
- **Semantic Mismatches:** Differences in what the data actually means (e.g., one component measures temperature in Celsius, the other in Fahrenheit).

Why they occur

- Components are built independently, by different teams or vendors, for different contexts
- Each component embeds **assumptions** about its environment that are rarely documented
- Components are often black boxes, so assumptions are discovered late
- Standards and technologies differ between components

Common Adaptation Techniques

1. Wrappers

1. A wrapper is a piece of code that **encloses a component and presents a new interface to the outside world**, while delegating the real work to the component inside. Clients talk only to the wrapper; the component stays untouched. It is the most common **black-box** adaptation technique, especially for COTS and third-party components.

- Think of it as a protective shell: whatever goes in or out passes through the wrapper, which can translate, filter, or enrich it

Why use a wrapper

- The source code is unavailable or must not be modified
- The component's interface doesn't match what the system expects
- You need to add behavior (security, logging, validation) the component lacks
- You want to **isolate vendor-specific details** so the component can be replaced later
- You need to expose a legacy component in a modern form (e.g., a COBOL routine as a REST service)

What a wrapper can do

- **Translate** interfaces, data, and protocols
- **Add** cross-cutting concerns such as security, logging, or error handling
- **Restrict** access to only the functions you want exposed
- **Combine** several calls into one simpler operation
- **Isolate** the system from changes in the component

Advantages and disadvantages

ADVANTAGES	DISADVANTAGES
No source changes needed	Extra layer adds some performance overhead
Preserves vendor upgrades and support	More code to write, test, and maintain
Isolates vendor lock-in	Cannot fix deep internal or architectural problems
Adds missing features such as security or logging	Hidden assumptions inside the component remain
Makes components replaceable	Many wrappers increase system complexity

Cont.: Common Adaptation Techniques

2. Superimposition / Aspect-Oriented Adaptation

- This is a more sophisticated, **"gray-box"** technique. It allows behavior to be "superimposed" onto a component. Instead of just wrapping the outside, the adaptation logic can intercept and modify messages at various points *within* the component's execution. This is useful for adding orthogonal concerns like logging, security, or fault tolerance across multiple method
- Superimposition is an adaptation technique where you **add new behavior on top of a component's existing behavior without changing its source code**, by "layering" extra functionality over it. The original component keeps working as before, and the superimposed code is woven in at defined points to modify, extend, or constrain what it does.
- The term comes from the academic literature on component adaptation (notably Bosch's work on "superimposition" as a way to adapt black-box components). It is conceptually close to **aspect-oriented programming (AOP)**.

Cont.: Common Adaptation Techniques

3 Adapter pattern

- The Adapter pattern lets two pieces of code work together even though their interfaces are incompatible. The adapter sits in the middle and **translates calls** from the interface the client expects (the *target*) into calls on the interface the component actually provides (the *adaptee*)
- In CBA, you usually cannot change a COTS or third-party component's source. The client code in your system already expects a particular interface. The adapter lets you:
- Keep the **client code unchanged**
- Keep the **component untouched** (so vendor upgrades and support still work)
- Isolate all translation logic in **one place**

Cont.: Common Adaptation Techniques

4. Configuration (Parameterization)

- Adjusting a component's behavior through **settings it already exposes**, rather than writing new code. The component was designed with variability in mind, and you simply select the options you need.
- **What you can adjust**
  - Connection details (database, server, ports)
  - Performance settings (cache size, thread pool, timeouts)
  - Security options (authentication mode, encryption level)
  - Logging level and format
  - Locale, currency, and date formats
  - Which optional features are active

Common form

FORM	EXAMPLES
Property / config files	application.properties, .ini, YAML, JSON, XML
Deployment descriptors	web.xml, Spring XML, Kubernetes/Docker config
Environment variables	Database URL, API keys, feature flags
Runtime parameters	Constructor arguments, setter methods, command-line flags
Admin consoles / GUIs	Vendor dashboards for turning features on or off
Feature toggles	Enable or disable optional modules

Cont.: Common Adaptation Techniques

5. Extension (Plug-ins and Extension Points)

- Adding **new behavior** to a component through hooks the vendor deliberately provides. You write your own code, but it plugs into the component through a defined interface, so the core remains untouched.
- is a mechanism for **extending system functionality through predefined "extension points"** , rather than modifying the original component's code. It is another pattern for component adaptation, but with more emphasis on **architectural-level extensibility**
- **Extension Point**: A predefined interface or contract declared by a host component/application, specifying "where extensions are allowed" and "what shape extensions must have
- **Extension (Plug-in)**: An independently developed module that implements the extension point contract and "plugs into" the host system to add functionalit
- The key is: the host component declares the extension point but **does not need to know who will extend it**, and the plug-in implements the extension point but **does not need to know the host's internal implementation**

Common extension mechanisms

MECHANISM	HOW IT WORKS	EXAMPLE
Plug-ins / add-ons	Self-contained modules loaded by the host	Browser extensions, WordPress plug-ins, IDE plug-ins
Callbacks / hooks	You register functions called at specific events	Webhooks, lifecycle hooks, event listeners
Interfaces to implement	Component calls your class through an agreed interface	Custom Validator or AuthProvider
Subclassing (frameworks)	Override designated methods	Overriding template methods
Scripting support	Embedded scripts alter behavior	Macros, rules engines
Middleware / interceptors	Code runs before or after component calls	Servlet filters, request interceptors
Custom APIs / SDKs	Vendor SDK for building add-on logic	Payment gateway SDKs

Cont.: Common Adaptation Techniques

5. ***Protocol adaptation***

- Adjusting communication procedures so components can exchange information using compatible protocols.
- Example: Using an adapter to allow a component that expects one communication protocol to interact with a service that uses another.

6. Refactoring / Re-engineering

This is a **white-box** technique that requires the component's source code. The component is structurally altered—for example, a monolithic component is "fragmented" into smaller, distributed parts to fit a new deployment architecture.

Challenges of Component Adaptation

- Adaptation may introduce new errors.
- It can increase system complexity.
- Some components may have limited documentation.
- Adaptation may reduce performance.
- Changes to a component may affect future updates or compatibility.
- The component's source code may not be available for direct modification.

End

!!!!!!!